

Systèmes d'exploitation II

Chapitre 2 : Synchronisation entre processus

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

The Big Picture

Linux kernel SCI (System Call Interface)

I/O subsystem

**Memory
management
subsystem**

**Process
management
subsystem**

Linux kernel
Virtual File System

Terminals

Sockets

File systems

Netfilter / Nftables

Network
protocols

Linux kernel
Packet Scheduler

Generic
block layer

Linux kernel
I/O Scheduler

Character
device
drivers

Network
device
drivers

Block
device
drivers

Virtual
memory

Paging
page
replacement

Page
cache

Signal
handling

process/thread
creation &
termination

Linux kernel
Process
Scheduler

IRQs

Dispatcher

Introduction

★ Dans un OS, plusieurs processus ou threads s'exécutent de manière concurrente, souvent en partageant des ressources (mémoire, fichiers, périphériques).

La **synchronisation** désigne l'ensemble des mécanismes permettant de coordonner l'exécution de processus ou de threads concurrents, afin de :

- garantir la cohérence des données partagées ;
- contrôler l'ordre d'accès aux ressources ;
- assurer des propriétés de sûreté (absence d'erreurs) et de progression (absence de blocage injustifié).

★ Une synchronisation incorrecte peut conduire à :

- des conditions de compétition (race conditions) ;
- des interblocages (deadlocks) ;
- de la famine (starvation).

➔ Le système fournit des primitives de synchronisation (verrous, sémaphores, verrouillage de fichiers, opérations atomiques) pour organiser l'accès concurrent aux ressources.

Sommaire

- 1 Condition de compétition, exclusion mutuelle & notion d'interblocage
- 2 Exclusion mutuelle par verrouillage de fichiers, attente active et sémaphores
- 3 Exclusion mutuelle par attente active
- 4 Schémas de synchronisation et sémaphores
- 5 Schémas de synchronisation

Objectifs

- identifier une **condition de compétition** et une **section critique** dans un programme concurrent ;
- expliquer le principe de l'**exclusion mutuelle** et ses contraintes ;
- reconnaître une situation d'**interblocage**, en comprendre les causes et distinguer **guérison** et **prévention** des interblocages ;
- comparer les principales **techniques de synchronisation** (attente active, verrous, sémaphores) ;
- utiliser des **sémaphores** pour résoudre des **schémas classiques de synchronisation** (exclusion mutuelle, cohorte, rendez-vous, producteurs/consommateurs, lecteurs/rédacteurs).

Problème de l'exclusion mutuelle

Agence A

```
1. current = get_account(1867A)
2. new = current + 10
3. update_account (1867A, new)
```

Agence B

```
1. current = get_account(1867A)
2. new = current - 10
3. update_account (1867A, new)
```

- Les instructions des deux processus peuvent être entremêlées de différentes façons par l'ordonnanceur :
 - ▶ A1 A2 A3 B1 B2 B3 → **compte inchangé**
 - ▶ A1 B1 A2 B2 A3 B3 → **compte -= 10**
 - ▶ B1 A1 B2 A2 B3 A3 → **compte += 10**
- Analyse :
 - ▶ les deux processus accèdent à une variable partagée (le solde).
 - ▶ les instructions sont exécutées de manière concurrente.
 - ▶ l'ensemble des instructions de mise à jour du solde constitue une **section critique (SC)**.
- On est en présence d'une **condition de compétition** (race condition)
- Une SC doit être exécuté en **exclusion mutuelle (EM)**

Condition de compétition (race condition)

On parle de **condition de compétition** lorsque le résultat d'un programme **dépend** de l'ordre d'exécution d'instructions **concurrentes** accédant à des ressources **partagées**.

- l'ordre d'exécution est imposé par l'ordonnanceur
 - il est généralement imprévisible.
- le comportement du programme devient **non déterministe**

★ Une condition de compétition apparaît lorsque les trois éléments suivants sont réunis :

- données partagées entre plusieurs processus ou threads ;
- exécutions concurrentes (entrelacement possible des instructions) ;
- absence de synchronisation pour contrôler l'accès à ces données.

→ Conséquences

- résultats incorrects ou incohérents ;
- bugs difficiles à reproduire et à corriger ;
- problèmes potentiels de sécurité (exploitation de fenêtres temporelles).

Section critique

Une **section critique** est une portion de code qui accède à une **ressource partagée** (donnée, fichier, périphérique, structure du noyau, ...) et dont l'exécution concurrente peut conduire à une condition de compétition.

→ Une section critique **doit être exécutée en EM** (protégée).

```
...  
protocole d'entrée en SC  
    section critique  
protocole de sortie de SC  
...
```

- le protocole d'entrée garantit l'EM
- le protocole de sortie libère la ressource

★ Un mécanisme de gestion des SC doit assurer :

- **Exclusion mutuelle** : Un seul processus dans la SC à un instant donné
- **Progression** : Si la SC est libre, un processus demandeur doit pouvoir y entrer
- **Absence de famine** : Aucun processus ne doit attendre indéfiniment l'accès à la SC

★ La SC ne doit contenir que le code **strictement nécessaire** pour réduire le temps de **blocage** et limiter les risques d'**interblocage** et de **famine**.

Réalisation de l'exclusion mutuelle : panorama

★ Comment garantir l'EM lors de l'accès à une SC, dans un système concurrent ?

→ Plusieurs familles de solutions existent, selon :

- ▶ l'architecture matérielle (mono / multiprocesseur),
 - ▶ le niveau (utilisateur / noyau),
 - ▶ les contraintes de performance et de sûreté.
- **Masquage des interruptions** à l'entrée de la SC, puis les restaurer à la sortie
 - ✓ Simple
 - ✗ Dangereux si les interruptions ne sont pas restaurées
 - ✗ Inefficace sur multiprocesseur (possibilité d'accès via un autre processeur)
 - **Attente active**: le processus boucle tant que la ressource est occupée.
 - ✓ Implémentable sans aide de l'OS
 - ✗ Gaspillage de CPU
 - **Primitives de synchronisation fournies par l'OS** - **ex.** sémaphores, Mutex / verrous, Verrouillage de fichiers, instructions atomiques
 - ✓ Solution générale et sûre
 - ✓ Adaptée aux systèmes multiprocesseurs
 - ✗ Mauvaise utilisation possible (deadlock, famine)

Notion d'interblocage

Situation :

2 processus P1 et P2

2 ressources R1 et R2

Processus P1

Acquire (R1) — ①

Acquire (R2) — ③

Use (R1, R2)

Release (R1)

Release (R2)

Processus P2

Acquire (R2) — ②

Acquire (R1) — ④

Use (R1, R2)

Release (R2)

Release (R1)

Ordre d'exécution : ① — ② — ③ — ④

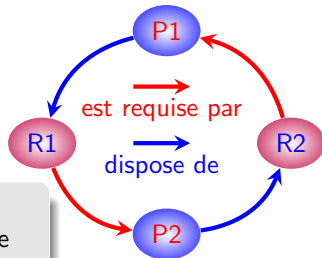
→ P1 et P2 sont bloqués *ad vitam eternam* :

- P1 attend Release(R2) de P2
- P2 attend Release(R1) de P1



→ C'est un **interblocage** (*deadlock*):

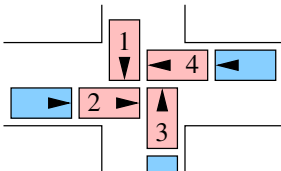
une situation dans laquelle un ensemble de processus **concurrents** attendent **mutuellement** la libération d'une ressource non partageable afin de pouvoir continuer à s'exécuter.



Présence d'un cycle dans la chaîne des attentes

Situation d'interblocage : analogie du carrefour

★ Un carrefour un vendredi, Paris à 18h



- quelles sont les ressources ?
- comment sortir de l'interblocage ?

★ Situation réelle d'interblocage :



Sortir de l'interblocage

guérison

- ★ Une fois un interblocage établi :
 - aucun processus ne peut progresser spontanément ;
 - le système doit intervenir pour casser le cycle d'attente.
- ➔ Toute solution a un coût.

Possibilités de guérison :

- **Terminaison de processus**
 - ▶ tuer un ou plusieurs processus (casser le cycle)
 - ✓ Simple
 - ✗ Perte du travail effectué
 - ✗ Qui tuer ?
- **Reprise arrière (rollback)**
 - ▶ forcer un processus à revenir à un état antérieur
 - ✓ Permet d'éviter la terminaison
 - ✗ Nécessite des points de reprise (checkpoints)
 - ✗ Perte du travail effectué depuis le dernier point de reprise

★ Il n'existe pas de solution idéale pour guérir un interblocage : toute correction entraîne une perte (temps, calcul, cohérence).

➔ **Qu'en est il de la prévention ?**



Comment prévenir l'interblocage ?

★ Réserve globale de toutes les ressources nécessaires
→ Damier jaune :

Processus P1

Acquire (R1 et R2)
utiliser (R1,R2)
Release (R2)
Release (R1)

Processus P2

Acquire (R1 et R2)
utiliser (R1,R2)
Release (R2)
Release (R1)



★ Requêtes ordonnées : tous les processus demandent les ressources dans le même ordre
→ Rond-point :

Processus P1

Acquire (R1)
Acquire (R2)
utiliser (R1,R2)
Release (R2)
Release (R1)

Processus P2

Acquire (R1)
Acquire (R2)
utiliser (R1,R2)
Release (R2)
Release (R1)

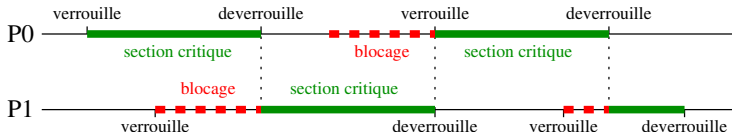


Exclusion mutuelle par verrouillage de fichiers

- ✓ Le **verrouillage** de fichiers consiste à utiliser un fichier comme support de **verrou** afin de garantir l'EM sur une ressource partagée (fichier lui même ou autre).
- ✓ Deux opérations **atomiques** sont fournies :
 - **verrouille**(fichier) : demande l'accès exclusif à fichier
 - ▶ Si fichier est libre → le verrou est accordé.
 - ▶ Si fichier est déjà verrouillé → le demandeur est bloqué jusqu'à sa libération.
 - **déverrouille**(fichier) : libère fichier ⇒ réveil d'un processus en attente du verrou (et un seul)
- ★ Le blocage est **passif** (pas d'attente active).

```
verrouille(fichier);  
accès à fichier (SC);  
déverrouille(fichier);
```

```
verrouille(fichier);  
accès à fichier (SC);  
déverrouille(fichier);
```



Verrouillage de fichiers : propriétés et sémantique

Types de verrous

- **Verrou exclusif**: un seul processus peut accéder au fichier → utilisé pour les op. d'écriture (exclusives)
- **Verrou partagé** : plusieurs processus peuvent accéder simultanément au fichier → utilisé pour les op. de lecture (concurrentes)

Verrou existant

Exclusif

Partagé

Demande exclusif

✗ bloqué

✗ bloqué

Demande partagé

✗ bloqué

✓ autorisé

Mode de verrouillage

- **Verrouillage bloquant** : Le processus est mis en attente tant que le verrou n'est pas disponible.
- **Verrouillage non bloquant (test)** La demande échoue immédiatement si le verrou est indisponible. Le processus décide alors quoi faire.

Verrouillage de fichier sous Unix

Le verrouillage peut être :

- **impératif** (*mandatory*) :
 - ▶ **bloque les accès** incompatibles avec verrous présents
rarement utilisé, non POSIX
- **consultatif** (*advisory*) :
 - ▶ n'empêche pas physiquement les accès concurrents.
 - ★ Panneau “ne pas entrer” visible,
les processus “bien élevés” respectent la règle ... (POSIX)

Deux primitives:

- **fcntl**: générale (verrous exclusifs et partagés, mode impératif ou consultatif, verrouillage de portions de fichier.) ✗ API complexe
- **lockf**: plus simple, mode consultatif, exclusif seulement avec possibilité de verrouiller fichier entier ou une partie

★ Tous les verrous d'un fichier sont de même type

Verrouillage de fichier sous Unix : lockf

```
#include <unistd.h>
int lockf(int fd, int cmd, off_t len);
```

- `fd` : descripteur du fichier à verrouiller
- `cmd` : commande de verrouillage
 - ▶ `F_ULOCK` : déverrouillage
 - ▶ `F_LOCK` : verrouillage exclusif **bloquant**
 - ▶ `F_TLOCK` : verrouillage exclusif **non bloquant** (test - l'appel échoue immédiatement si le verrou est indisponible, retourne une erreur)
 - ▶ `F_TEST` : test de disponibilité du verrou
- `len`: portion du fichier à verrouiller :
 - ▶ `= 0` : fichier complet
 - ▶ `> 0` : `len` octets après la position courante
 - ▶ `< 0` : taille octets avant la position courante
- **Valeur de retour** 0 si succès, -1 sinon (*cf. errno*)

Verrouillage de fichier sous Unix : exemple

testlock.c

```
int fd = open("toto", O_RDWR);
while (1) {
    if (lockf(fd, F_TLOCK, 0) == 0) {
        printf("%d lock\n", getpid());
        sleep(5);
        if (lockf(fd, F_ULOCK, 0) == 0)
            printf("%d unlock\n", getpid());
        return;
    } else {
        printf("%d locked !\n", getpid());
        sleep(2);
    }
}
```

```
./testlock & ./testlock &
[2] 152994
152995 lock
152994 locked !
152994 locked !
152994 locked !
152995 unlock
$ 152994 lock
152994 unlock
```

L'exemple des banques revisité

Rappel du problème

Agence A

```
current = get_account(1867A)
new = current + 10
update_account (1867A, new)
```

Agence B

```
current = get_account(1867A)
new = current - 10
update_account(1867A, new)
```

Implémentation avec le verrouillage de fichier UNIX

Agence A

```
int fd = open("fichier partagé", O_RDWR);
lockf(fd, T_LOCK, 0);
current = get_account(1867A)
new = current + 10
update_account (1867A, new)
lockf(fd, T_UNLOCK, 0);
```

Agence B

```
int fd = open("fichier partagé", O_RDWR);
lockf(fd, T_LOCK, 0);
current = get_account(1867A)
new = current - 10
update_account(1867A, new)
lockf(fd, T_UNLOCK, 0);
```

Section critique par attente active

★ Principe : On demande tant qu'on a pas obtenu

```
while (ressource non libre){ }; //attente active en entrée
ressource libre = false;
utiliser ressource; //section critique
ressource libre = true; // sortie de la SC
...
```

race condition !!

✗ Manque d'efficacité car gaspillage de ressources

✓ Implémentable sans primitive de l'OS ni du matériel

→ Utilisée pour des verrous de très courte durée **ex. spinlock** (Linux) : moins "cher" pour le système que de subir un changement de contexte.

Réalisation d'une SC par attente active

★ **Hypothèses:** atomicité d'accès et de modifications de variables

V ne change pas au milieu de $V==1$ ni de $V=1$ (raisonnable sur monoprocesseur)

Toutes fausses !!

1. Alternance stricte

```
while (tour != MOI); /* attente active */
SC();                /* section critique */
tour = 1-MOI;        /* soyons fair-play */
```

✗ Entrer deux fois de suite dans SC() impossible (y compris avec un processus seul)

→ **Famine**

2. Drapeaux simples

```
while(flag[1-MOI]); /*attendre autre*/
flag[MOI] = VRAI;   /*annonce entrer*/
SC();
flag[MOI] = FAUX;   /*débloque autre*/
```

✓ P0 teste flag[1] et trouve faux

✗ P1 teste flag[0] et trouve faux

✗ Les deux entrent dans SC()

→ **Pas d'EM (compétition):**

3. Annonce puis attente

```
flag[MOI]=VRAI; /*annonce entrer avant*/
while (flag[1-MOI]); /*attendre l'autre*/
SC();
flag[MOI] = FAUX; /*débloque autre*/
```

✓ P0: flag[0] ← VRAI

✓ P1: flag[1] ← VRAI

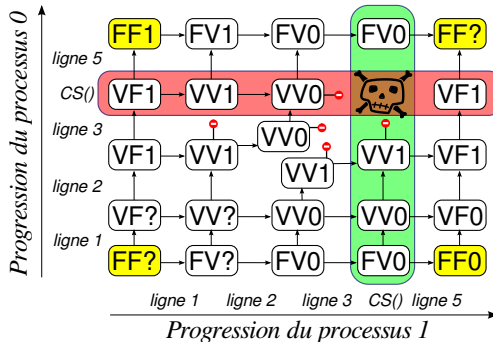
✗ Les deux entrent dans leur boucle

→ **Possibilité d'interblocage:**

Algorithme correct d'attente active [Peterson-83]

```

1.  flag[MOI] = VRAI;                                /* Annonce être intéressé */
2.  tour = 1-MOI;                                     /* mais on laisse la priorité à l'autre */
3.  while ((flag[1-MOI]==VRAI)&& (tour == 1-MOI));
/* on entre si l'autre ne veut pas entrer ou s'il nous a laissé la priorité */
4.  CS();
5.  flag[MOI] = FAUX;                                /* release */
    
```



★ Ceci est un **diagramme de transition** (rarement aussi régulier)

Algorithme de Peterson : propriétés et limites

Propriétés garanties (2 processus) l'algorithme de Peterson assure :

- **Exclusion mutuelle** Les deux processus ne peuvent jamais être simultanément en section critique.
- **Progression** Si un processus souhaite entrer et que l'autre n'est pas intéressé, il entre.
- **Absence de famine** En cas de conflits répétés, la priorité alterne équitablement.

Hypothèses nécessaires

- Accès **atomiques** aux variables partagées.
- Ordre de visibilité cohérent des écritures (modèle mémoire simple).
- Nombre de processus strictement égal à 2.
 - ★ algorithme du boulanger [Lamport] une généralisation classique à N processus

Limites pratiques

- ne passe pas à l'échelle (généralisation complexe).
- attente active → gaspillage de CPU.

Principaux problèmes de synchronisation

- **Condition de compétition** (*race condition*) : le résultat dépend de l'ordre d'exécution des instructions concurrentes
 - ▶ **Cause** : données partagées, exécution concurrente, absence de synchronisation.
 - ▶ **Effets** : résultats incohérents, bugs difficiles à reproduire, failles de sécurité :
 - ★ un programme crée un fichier temporaire, le remplit puis utilise le contenu
 - ★ l'attaquant crée le fichier avant le programme pour contrôler le contenu
 - **Interblocage** (*deadlock*) : ensemble de processus bloqués en attente mutuelle de ressources non partageables.
 - ▶ **Cause principale** : attente circulaire entre processus et ressources.
 - ▶ **Effet** : pas de progression spontanée.
 - ▶ **Solutions** :
 - ★ guérison (terminaison, rollback),
 - ★ prévention (ordre global, réservation).
 - **Famine** (*starvation*) : un processus attend indéfiniment une ressource
 - ▶ Absence d'équité dans l'allocation.
 - ▶ Peut apparaître même sans interblocage.
- ➔ La synchronisation vise à garantir la **cohérence**, éviter **rares**, **deadlocks** et **famine**, tout en préservant les **performances**.

Schémas de synchronisation

- 1 **Exclusion mutuelle** : une ressource n'est utilisée que par un seul processus à la fois.
 - ✓ carte son, compte bancaire, toilettes individuelles, cabine d'essayage, ...
- 2 **Cohorte** : une ressource peut être partagée par **au plus N processus simultanément**
 - ✓ serveur (N connexions), parking (N places), piscine avec jauge maximale ...
- 3 **Rendez-vous** : des processus doivent **s'attendre mutuellement** avant de continuer.
 - ✓ un processus devant échanger des informations entre les étapes de l'algorithme
 - ✓ 2 personnes qui se donnent rendez-vous avant de partir
 - ✓ signature d'un contrat : les deux parties doivent être présentes
- 4 **Producteurs/Consommateurs** : un processus **doit attendre** qu'un autre ait produit une information
 - ✓ réception de données réseau **puis** traitement
 - ✓ le facteur ne peut récupérer les lettres qu'après leur dépôt dans la BaL
- 5 **Lecteurs/Rédacteurs** : accès **concurrent** pour une catégorie, **exclusif** pour l'autre.
 - ✓ une section de voie unique : tous doivent rouler dans le même sens
 - ✓ un fichier : lectures concurrentes, écriture exclusive.

Schémas de synchronisation

Outil générique ?

- ✓ Comment résoudre tous ces schémas correctement, sans attente active, avec des garanties de synchronisation ?
- Il existe un outil générique et puissant capable de résoudre tous ces schémas :

Les **sémaphores** permettent de :

- contrôler l'accès à une ressource,
- synchroniser des événements,
- bloquer et réveiller proprement les processus.

★ Un même mécanisme, des usages multiples.

Sémaphore : outil de base pour la synchronisation

Un **sémaphore** est un objet composé :

- une **variable entière** : nombre de permissions disponibles ;
- une **file d'attente** : processus bloqués sur le sémaphore.

✓ Primitives **atomiques** associées :

- **Initialisation** : valeur initiale ≥ 0 ;
- **P (Probeer, wait, down)** - demande d'autorisation (" puis-je ?") :
si $valeur > 0 \Rightarrow valeur = valeur - 1$;
sinon $\Rightarrow$ **blocage** du processus.
- **V (Verhoog, signal, up)** - fin d'utilisation (" vas-y") :
si des processus sont bloqués $\Rightarrow$ **réveil de l'un d'eux** ;
sinon $\Rightarrow valeur = valeur + 1$.

1. Exclusion mutuelle par sémaphore

Très simple

Initialisation

```
sem=semaphore(1)
```

Agence A

```
P(sem)  
current = get_account(1867A)  
new = current + 10  
update_account (1867A, new)  
V(sem)
```

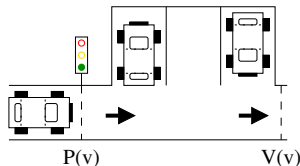
Agence B

```
P(sem)  
current = get_account(1867A)  
new = current - 10  
update_account (1867A, new)  
V(sem)
```

2. Cohortes et sémaphores

Sémaphore inventé pour cela

- * Cohorte : généralisation de l'exclusion mutuelle
- Ressource partagée par au plus N utilisateurs
 - ✓ Parking à N places
 - ✓ Serveur web à N connexions simultanées



Initialisation

```
sem=semaphore(3)      /* nombre de places */
```

Garer sa voiture

```
P(sem)
```

```
poser sa voiture au parking
```

```
aller faire les courses
```

```
Reprendre la voiture
```

```
V(sem)
```

```
partir
```

3. Rendez-vous et sémaphores

- ✓ **Envoi de signal** : un processus indique quelque chose à un autre (donnée disponible)

Initialisation

```
top=semaphore(0)
```

Processus 1

```
calcul(info)  
V(top)
```

Processus 2

```
P(top)    /*Bloque en attente*/  
utilisation(info)
```

- top est un **sémaphore privé** (threads d'un processus), initialisée à 0, il ne protège pas une ressource, il synchronise un événement.

- ✓ **Rendez-vous entre deux processus** : les processus s'attendent mutuellement

Initialisation

```
roméo=semaphore(0)  
juliette=semaphore(0)
```

Processus Roméo

```
P(romeo)    /*se bloque*/  
V(juliette) /*libère J*/
```

Processus Juliette

```
V(romeo)    /*libère R*/  
P(juliette) /*bloque*/
```

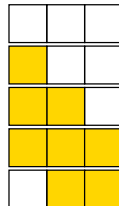
- ✓ **Rendez-vous entre plus de 2 processus** : **barrière de synchronisation**
 - la solution précédente est généralisable, mais un peu lourde
 - souvent une primitive du système

4. Producteurs / Consommateurs et sémaphores

Schéma de contrôle de flux classique (par tampon)

★ Illustration.

- Situation initiale : tampon vide (pas de lecture possible)
- Après une production:
- Une autre production:
- Encore une production: (plus de production possible)
- Une consommation: (production de nouveau possible)



➔ L'idée est d'utiliser deux sémaphores pour gérer les deux états limites (plein et vide)

Initialisation

```
placeDispo=semaphore(N)  
infoPrete=semaphore(0)
```

Producteur

```
répéter  
  calcul(info)  
  P(placeDispo)  
  déposer(info)  
  V(infoPrete)
```

Consommateur

```
répéter  
  P(infoPrete)  
  extraire(info)  
  V(placeDispo)  
  utiliser(info)
```

- ★ Le tampon doit être circulaire pour traiter données dans l'ordre de production
- ★ Attention aux compétitions entre producteurs (idem entre consommateurs)

5. Lecteurs/Rédacteurs et sémaphores

Contrôle d'accès entre *catégories* d'utilisateurs

- Accès simultané de plusieurs lecteurs
- Accès exclusif d'un seul rédacteur, également exclusif de tout lecteur
- Schéma assez classique (fichier sur disque, zone mémoire, etc.)

Initialisation

```
NbLect=0  
rédacteur=semaphore(1)  
lecteur=semaphore(1)
```

Lecteur

```
P(lecteur)  
NbLect = NbLect + 1  
si NbLect == 1 alors  
    P(rédacteur)  
V(lecteur)  
lectures()  
P(lecteur)  
NbLect = NbLect - 1  
si NbLect == 0 alors  
    V(rédacteur)  
V(lecteur)
```

Rédacteur

```
P(rédacteur)  
lecturesEcritures()  
V(rédacteur)
```

Lecteurs et rédacteurs sans famine

Initialisation

```
lecteur=semaphore(1)
rédacteur=semaphore(1)
fifo=semaphore(1)
NbLect=0
```

Lecteur

```
P(fifo)
P(lecteur)
NbLect = NbLect + 1
si NbLect == 1 alors
    P(rédacteur)
V(lecteur)
V(fifo)
lectures()
P(lecteur)
NbLect = NbLect - 1
si NbLect == 0 alors
    V(rédacteur)
V(lecteur)
```

Rédacteur

```
P(fifo)
P(rédacteur)
V(fifo)
lecturesEtEcritures()
V(rédacteur)
```

Récap. Synchronisation des processus

Problèmes à éviter lors des synchronisations

- **Interblocage**: groupe de processus en attente mutuelle
- **Compétition**: le résultat dépend de l'ordre d'exécution
- **Famine**: un processus n'obtient pas la ressource car les autres l'en empêchent

Les sémaphores

- Principe: distributeur de jetons
- Opérations:
 - ▶ initialisation: mettre des jetons dans la boîte
 - ▶ P: prendre un jeton (ou bloquer s'il y en a plus)
 - ▶ V: rendre un jeton pris

Schémas de synchronisation classiques

- Exclusion mutuelle: Ressource utilisée par au plus un utilisateur
- Cohorte: Ressource partagée entre au plus N utilisateurs
- Rendez-vous: un processus attend un autre, ou attente mutuelle
- Producteurs/consommateurs: utilisation **après** création
- Lecteurs/rédacteurs: concurrence entre **catégories** d'utilisateurs